Rev 1.2 20160219

Guía de estilo de programación en Java

Pequeños consejos sobre cómo escribir programas más legibles.

Índice

Guía de estilo de programación en Java....1
1Introducción....3
2Indicaciones generales....3
3Identificadores....5
3.1Identificadores de clases....5
3.2Identificadores de atributos y variables locales....6
3.3Identificadores de constantes....6
3.4Identificadores de objetos....7
3.5Identificadores de métodos....7
3.5.1Identificadores de variables locales en métodos....7
3.6Definición de variables locales....8
4Comentarios....9
4.1Comentarios de inicio de bloque....9
4.2Comentarios aclaratorios....10
4.3Comentarios sobre métodos....10
5Disposición de elementos....11
5.1Clases....11
5.2Disposición de elementos en cada fichero....12
5.3Métodos....14
5.3.1Estructuras de decisión y repetición....15
6Bibliografía....16

1 Introducción

En este documento se explica brevemente, y con ejemplos, cómo escribir programas más sencillos de entender. A lo largo de su vida profesional, cualquier programador se plantea la pregunta de cómo escribir sus programas de manera que sean más legibles, y por tanto que puedan ser mantenidos (corrección de errores, ampliación de funcionalidad) con facilidad, tanto por él mismo como por otros programadores (una situación típica en cualquier empresa).

Dado que la misma situación en cuanto a legibilidad de código suele ser resuelta por distintos programadores de la misma forma, pero con distintos matices de diferenciación, es interesante seguir una guía de estilo que nos explique como otros resolvieron ese mismo problema, hasta que dicha solución se convirtió en un estándar.

2 Indicaciones generales

El propósito de seguir una norma de estilo es hacer que el código fuente de un programa sea tan legible como sea posible. Así, hay tres puntos básicos que se deben cuidar: el espaciado horizontal, el espaciado vertical y la indentación. El espaciado horizontal consiste en que las líneas en las que esté dividido el código fuente deben ser de una longitud máxima lo más próxima posible a 70 caracteres, ya que al imprimir el código en papel cualquier línea superior a 80 columnas se corta automáticamente.

Uno de los problemas que se puede encontrar al codificar es la manera más correcta para dividir una línea muy larga. Respuestas comunes a esta pregunta son: después de una coma (en una llamada a un método, por ejemplo), antes de una subexpresión, antes de un operador, o antes de un paréntesis. Por ejemplo:

```
int x = ( (a * b + c ) / ( c * d * d ) )
        + ( a / ( b * c ) )
        + ( ( 3.1451927 * b ) + d );
```

Se recomienda separar las palabras reservadas (*if*, *while*...), los operadores, así como los paréntesis, mediante espacios para mayor claridad, a excepción hecha de operadores unarios como el incremento o decremento, tanto postfijo como prefijo, que suelen colocarse pegados al operando al que modifican. El uso típico incluye además el separar con un espacio los argumentos en las listas de parámetros formales y de argumentos, los corchetes del operador de subíndice, y los operadores como =, *. /, + y -.

```
int x = a + b;
int z = Math.pow( x, y );

numRegistros = v.size() + registrosCabecera[ i ];

if ( x > 0 ) {
      // más cosas...
}
```

El espaciado vertical consiste en cuántas líneas ocupa un método, o una clase, en el código fuente. En general, es útil autoimponerse un límite de una hoja por cada método. Es extraño, siempre que la función miembro no consista en multitud de acciones repetitivas (como escribir en un archivo, o realizar una serie de incializaciones), que una función miembro bien diseñada ocupe más allá de un folio. En caso contrario, será conveniente considerar que posiblemente sea interesante subdividirlo en varios métodos privados.

Además, en el espaciado vertical intervienen las llaves que se emplean para marcar el comienzo y fin del cuerpo de una función, de un bucle,... cada una de esas llaves puede llegar a consumir una línea por sí sola, incluso dos. Así, para salvar algo de espacio vertical, se suelen mantener las llaves de apertura y cierre, cada una en una línea, para el cuerpo de las funciones miembro y funciones, y en cambio colocar la llave de apertura en la misma línea para bucles y estructuras de decisión (a esto se le conoce como estilo *Kernighan&Ritchie*). Por ejemplo:

```
public class Mates {
	public static int divide(int a, int b)
	{
		if ( b != 0 ) {
			return a / b;
		}

		return 0;
	}
	// más cosas...
}
```

(...si bien en un programa real será mucho más adecuado responder a la contingencia que supone dividir un número entre cero, que responder con cero “silenciosamente”, lo cual supone un error no detectado.)

Así, por regla general la llave de apertura se coloca en la misma línea, siempre y cuando no se trate de la llave de apertura de un método, o que en una sentencia *if* se encuentren varias subcondiciones.

La indentación es vital para poder comprender rápidamente los elementos funcionales de un programa. Así, el cuerpo (las instrucciones) de una función o función miembro debe estar alineado en su margen izquierdo más a la derecha que la cabecera de una función, de la misma forma que un bucle dentro de esa función y así sucesivamente. Por ejemplo:

```
public class Mates {
	public static int eleva(int x, int y)
	{
		int toret = x;

		for(; y > 1; --y) {
			toret *= x;
		}

		return toret;
	}
}
```

3 Identificadores

Los identificadores se utilizan en varias (innumerables) ocasiones en programación: clases, atributos, métodos, constantes, variables locales,... y son claves para entender qué valores puede contener una variable, qué hace una función miembro o a qué objetos representa una clase. Por eso, hay que poner especial cuidado en su elección. También será interesante seguir unas pequeñas reglas a la hora de escribir un identificador que hagan que a su vez podamos obtener el máximo significado del mismo si se está leyendo en un listado.

En general, los identificadores deben ser tan cortos como sea posible, pero a la vez tan informativos como sea posible. En general, las clases, atributos y variables locales se nombrarán con sustantivos (**Coche**, **Casa**), mientras para los métodos se utilizarán verbos en su forma imperativa (*divide*, *compara*). Muchas veces, además, es imposible utilizar un solo sustantivo para nombrar una clase, atributo, o variable local; en ese caso, se concatenarán todos para formar el identificador final, poniendo cada inicial en mayúscula. Si bien algunos lenguajes modernos lo permiten (como Java), a través del soporte **unicode**, evítense los acentos, las diéresis... en los identificadores.

3.1 Identificadores de clases

Los identificadores de clases deberían corresponderse con sustantivos de la vida real o del concepto que se está modelando con el programa. Así, identificadores de clases pueden ser: **Casa**, **Coche**, **Barco**, **Cuenta**,...

El identificador de la clase, en caso de estar compuesto por más de una palabra, se construye concatenando todas las palabras, y poniendo la inicial de cada una de estas palabras en mayúsculas. Por ejemplo: **CuentaCorriente**, **VehiculoConMotor**, **VehiculoSinMotor**.

```
public class Rectangulo {
      // más cosas ...
}

public class Circulo {
      // más cosas ...
}

public class Coche {
      // más cosas ...
}

public class CuentaCorriente {
      // más cosas ...
}
```

3.2 Identificadores de atributos y variables locales

Los identificadores de datos miembro siguen las mismas normas que los de las clases, pero con la primera inicial en minúscula. Así, por ejemplo, identificadores válidos son *identificadorCompleto*, *precioEuros*, ...

```
public class CuentaCorriente {
      private double saldoEuros;

      public CuentaCorriente(double saldo)
      {
            saldoEuros = saldo;
      }

      public double getSaldo()
      {
            return saldoEuros;
      }

      // más cosas ...
}
```

3.3 Identificadores de constantes

Los identificadores de constantes siguen un formato muy especial: siempre en mayúsculas, y en caso de contener varios nombres, estos se concatenan utilizando el carácter subrayado ('_'). Esto incluye las constantes de los enumerados.

```
public class CuentaCorriente {
      public static final double EURO_A_PESETAS = 166.386;
      public enum TipoCuenta { AHORRO, AHORRO_INVERSION_BOLSA };

      private double saldoEuros;
      private TipoCuenta tipoCuenta;

      public CuentaCorriente(double saldo, TipoCuenta tipo) {
            saldoEuros = saldo;
            tipoCuenta = tipo;
      }

      public TipoCuenta getTipoCuenta() {
            return tipoCuenta;
      }

      public double getSaldo() {
            return saldoEuros;
      }


      public double calculaSaldoEnPesetas() {
            return getSaldo() * EURO_A_PESETAS;
      }

      // más cosas ...
}
```

3.4 Identificadores de objetos

Los identificadores de objetos siguen las mismas reglas que los identificadores de atributos y variables locales.

```
public class Ppal
      public static void main(string[] args)
      {
            CuentaCorriente cuenta = new CuentaCorriente( 500,
                                     CuentaCorriente.TipoCuenta.AHORRO );

            System.out.println( "Saldo: " + cuenta.getSaldo() );
      }
}
```

3.5 Identificadores de métodos

Los identificadores de funciones miembro siguen las mismas reglas de mayúsculas y minúsculas que para atributos, variables locales y objetos. Sin embargo, deben escogerse verbos de manera que sugieran de manera intuitiva qué hacen. Así, el identificador debe ser un verbo en imperativo o al menos contener uno. Los métodos que sirven para obtener el valor de un atributo deben llevar el prefijo *get*.

```
int getEdad();
boolean esPalindromo(String s);
```

Evítensen identificadores como los siguientes:

```
int procesar(File f);              // Mal: identificador no intuitivo
String pasoAuxiliar(String aux);   // Mal: identificador erróneo
void procesaYCuenta();             // Mal: Dividir en dos funciones
```

Cuando un identificador contiene una conjunción como *y*, es signo inequívoco de que la función que nombra realiza más de una tarea y debe ser por tanto separada en dos funciones separadas.

Los mejores identificadores son aquellos que describen con un identificador más corto, y a la vez de manera más específica lo que hace el método. Además, es interesante seguir ciertas pautas: en el caso de funciones miembro que devuelven un booleano (un valor verdadero o falso), es interesante nombrarlos con un prefijo formado por los verbos ser o estar, como:

```
boolean esPalindromo(String s);
boolean esPar(int x);
boolean fueModificada(Persona p);
```

3.5.1 Identificadores de variables locales en métodos

En el caso de variables locales de funciones miembro, existen varias particularidades. Por ejemplo, a las variables locales empleadas en bucles se les suele asignar identificadores de una letra tipo 'i' y 'j', también en el caso de algunos argumentos simples de métodos (como 's' o 'x', si bien este caso es mejor, sin embargo, evitarlo, cuando sea posible, y asignar identificadores descriptivos).

```
public class Cadenas {
      public static String capitalizaInicial(String s)
      {
            return s.toUpperCase().charAt( 0 )
                        + s.toLowerCase().substring( 1 );
      }
}
```

Los identificadores de variables también pueden informar sobre para qué se utiliza esa variable, y no restringirse a tan solo información sobre qué valores alberga. Por ejemplo, en el código siguiente *toret* (a retornar) es una variable que se utiliza en todas las funciones para devolver un valor.

```
public class Cadenas {
      public static String extraeConsonantes(String s)
      {
            final String vocales = "aeiou";
            StringBuilder toret = new StringBuilder();

            for (int i = 0; i < s.length(); ++i) {
                  final char ch = s.charAt( i );

                  if ( vocales.indexOf( ch ) < 0 ) {
                        // No encontrado en vocales, insertar
                        toret.append( ch );
                  }
            }

            return toret.toString();
      }
}
```

3.6 Definición de variables locales

Se debe colocar cada variable en una línea, incluso siendo del mismo tipo, para maximizar la legibilidad. Las variables locales siempre se colocan al comienzo de cada bloque, que siempre va delimitiado por llaves ('{' y '}'). También debe tratarse de inicializar las variables lo antes posible.

```
public class Ppal {
      private int x, y;             // MAL

      public static void main(String[] args)
      {
            int x = 0;
            int y = 1;
            char c, c2;             // MAL
            String toret = "";

            // más cosas ...
      }
}
```

4 Comentarios

Un comentario debe ser siempre clarificador, útil, y, en cambio, cuanto más corto mejor. En particular, debe cuidarse en no insultar la inteligencia del lector en determinadas ocasiones, comentando secuencias de código obvias y desesperarlo al encontrarse con construcciones complejas que no tienen ningún comentario.

```
int areaRectangulo = lado1 * lado2;       // calcula área
areaCirculo = PI * r * r;                 // calcula área del círculo
                                          // PI es 3.1415927
```

En el contexto del ejemplo anterior, el tercer comentario es absolutamente innecesario, mientras que los dos primeros son cuestionables, siempre que los identificadores hayan sido escogidos cuidadosamente, como es el caso. Cuando los identificadores son tan significativos que el propósito es obvio, el código se conoce como *autocomentado*. Esto no quiere decir que los comentarios sean siempre opcionales.

```
int areaRectangulo = lado1 * lado2;       // calcula el área en cms^2
```

4.1 Comentarios de inicio de bloque

Existen dos tipos básicos de comentarios, los que podríamos denominar *comentarios encima*, y los que podríamos denominar *comentarios a la derecha*. Los más recomendables son los primeros, pues suelen explicar un bloque de código, a modo de párrafo, aclarando mucho la lectura.

```
public class Persona {
      public Persona(String n, int e)
      {
            nombre = n;
            edad = e;
      }

      public String getNombre()
      {
            return nombre;
      }

      public int getEdad()
      {
            return edad;            // En años
      }

      public String toString()
      {
            // Formatear nombre
            String nombre = Cadenas.capitalizaInicial( getNombre() );

            // Construir info de persona
            String toret = getNombre() + ": " + getEdad();

            return toret;
      }
}
```

4.2 Comentarios aclaratorios

Los comentarios *a la derecha* deben emplearse como mensajes aclaratorios, intentando mantener especialmente en ellos la concisión, pues es fácil que se alcancen rápidamente más de ochenta caracteres en esa línea. Agravando aún más este último problema, deben colocarse alejados del código que aclaran para que sean visibles.

```
// Cálculos previos al rendering
areaRectangulo = lado1 * lado2;                // en cms
```

4.3 Comentarios sobre métodos

Desde la llegada del lenguaje Java, y su herramienta de documentación, *Javadoc*, se han generalizado los comentarios formateados con un determinado estilo, y aceptando unos determinados parámetros de documentación embebidos en ellos.

Así, cuando un comentario, en lugar de empezar por /*, comienza por /**, al pasar el código por la herramienta *Javadoc* (muchos entornos de programación ya lo hacen automáticamente), ésta genera la documentación recogida en formato HTML. Este formato es ideal, por sus hiperenlaces, para manejar referencias cruzadas entre miembros de clases.

Aún sin generar HTML estos comentarios también son útiles al desnudo, por lo que deben colocarse especialmente en aquellos miembros públicos. Es la interfaz que será consultada por el programador que utilice la clase para poder emplearla en su código.

```
/** Representa a los rectángulos */
class Rectangulo {
      /** La información sobre la base del rectángulo */
      private double base;

      /** La información sobre la altura del rectángulo */
      private double altura;

      /**
       * Constructor de rectángulos
       * @param b La base de la figura
       * @param a La altura de la figura
       */
      public Rectangulo(double b, double a)
      {
            base = b;
            altura = a;
      }

      /**
        * Devuelve la base de la figura
        * @return la base de la figura, como double
        */
      public double getBase()
      {
            return base;
      }
```

```
        /**
          * Devuelve la altura de la figura
          * @return la altura de la figura, como double
          */
        public double getAltura()
        {
              return altura;
        }

        /**
         * Calcula la superficie de la figura
         * @return La superficie de la figura, como double
         */
        public double calculaArea()
        {
              return getBase() * getAltura();
        }
}
```

De los parámetros que se pueden utilizar en este tipo de comentarios, destacan *@param* y *@return*. El primero sirve para documentar un parámetro de una función miembro. tal y como se ve en el constructor de la clase **Rectangulo** de ejemplo, más arriba. El segundo sirve para documentar el valor de retorno de una función miembro, tal y como se aprecia en la función miembro `calculaArea()` de la clase del mismo ejemplo.

5 Disposición de elementos

5.1 Clases

Debe escogerse una forma de disponer los miembros en la clase, y tratar de mantenerla. Por ejemplo, la más extendida consiste en colocar la parte pública de la clase lo más cercana a la parte superior. Esto tiene sentido porque cuando trate de leer el código, al programador le interesa la parte pública de la clase, que es la que tendrá que manejar, y no la parte privada o protegida, de la que sólo tendrá que preocuparse si es el que la mantiene.

Java tiene, sin embargo, su propia forma de situar los miembros de la clase, que describe en su guía de estilo. El esquema que propone es colocar primero el grupo de atributos y después el grupo de métodos. Dentro de cada grupo irían primero los miembros públicos, después los protegidos, y finalmente los privados.

```
/** Representa a los puntos de dos dimensiones */
class Punto {
      private int x;
      private int y;

      /**
        * Constructor de objetos punto
        * @param a La coordenada horizontal
        * @param b La coordenada vertical
        */
      public Punto(int a, int b)
      {
            x = a;
            y = b;
      }

      /** @return la coordenada horizontal */
      public int getX()
      {
            return x;
      }

      /** @return la coordenada vertical */
      public int getY()
      {
            return y;
      }
}
```

5.2 Disposición de elementos en cada fichero

Cada fichero (*.java) en **Java** debe contener exactamente una clase, y el nombre del archivo debe coincidir con esa clase.

Lo primero que debe aparecer en el archivo es un comentario describiendo la información de versión de la clase. A continuación, la sentencia *package*, si es necesaria. Después los *import* que se utilicen, y a continuación la clase en sí.

```
/*
 * Punto
 * Fecha: 2014-12
 * Licencia MIT
*/

/** Representa a los puntos de dos dimensiones
  * @author: Baltasar
*/
public class Punto {
      public static final Punto ORIGEN = new Punto( 0, 0 );

      private int x;
      private int y;

      /**
        * Constructor de objetos punto
        * @param a La coordenada horizontal
        * @param b La coordenada vertical
        */
      public Punto(int a, int b)
      {
            x = a;
            y = b;
      }

      /** @return la coordenada horizontal */
      public int getX()
      {
            return x;
      }

      /** @return la coordenada vertical */
      public int getY()
      {
            return y;
      }

      /** @return la info del punto, como String */
      public String toString()
      {
            return ( getX() + ", " + getY() );
      }
}
```

Dentro de la clase, los elementos se disponen de la forma siguiente: primero las constantes (atributos marcados como final), los miembros *static*, a continuación los atributos, y después el constructor y los métodos. El orden dentro de cada "sección" siempre es el mismo, de *public* a *private*. Cada sección se separa de la anterior por una línea en blanco. Cada método se separa del anterior por una línea en blanco.

5.3 Métodos

En las funciones, se debe adoptar como esquema básico la típica estructura inicialización-desarrollo-limpieza, es decir el comienzo de la tarea, inicializando (y creando, cuando sea oportuno) las variables necesarias, el desarrollo del problema, como el bucle del ejemplo situado más abajo,y finalmente, cuando es necesario, la limpieza de los recursos utilizados.

```
public class ArrayUtil {

      /**
        * Concatena dos vectores de enteros
        * @param v1 El primer vector de enteros
        * @param v2 El segundo vector de enteros
        * @returns un nuevo vector de enteros: v1 + v2
        */
      public static int[] concatenaVectores(int[] v1, int[] v2)
      {
            int[] toret = new int[ v1.length + v2.length ];

            // Copia el primer vector en el resultado
            for(int pos = 0; pos < v1.length; ++pos) {
                  toret[ pos ] = v1[ pos ];
            }

            // Copia el segundo vector en el resultado
            for(int pos = 0; pos < v2.length; ++pos) {
                  toret[ pos + v1.length ] = v2[ pos ];
            }

            return toret;
      }
}
```

5.3.1 Estructuras de decisión y repetición

Es típico encontrarse con la necesidad de tener que crear estructuras condicionales (*if*) o de repetición (*while*), que se refieren a condiciones complejas. Debe tratarse, en estos casos, de disponer una subcondición por línea, comenzando por el juntor (*and* (&&), *or*(||), y *not*(!)). Si es necesario, una subcondición puede llevar un comentario "a la derecha". Si existen varias subexpresiones condicionales, se deben indentar respecto a la expresión principal.

```
 if ( v1.length > 0
    && v2.length > 0 )
{
      int pos = 0;

      while( pos < v1.length ) {
            // más cosas ...
      }

      if ( pos >= v1.length
        && pos >= v2.length )
      {
            v2[ pos ] = v1[ pos - v2.length ];
      }
}
```

Nunca deben crearse bloques que no consten de una llave de comienzo y otra de final.

```
      if ( v1.length > 0
        && v2.length > 0 )
            v2[ 0 ] = v1[ 0 ];                              // MAL
```

6 Bibliografía

1. **Kernighan, Ritchie (1991)**. *El lenguaje de Programación C*. Prentice-Hall.

2. Guía de estilo de programación en Java de Oracle
   http://www.oracle.com/technetwork/java/javase/documentation/codeconvtoc-136057.html

3. Guía de estilo de programación en Java de Google
   https://google-styleguide.googlecode.com/svn/trunk/javaguide.html